HABIB UNIVERSITY

Data Structures & Algorithms

CS/CE 102/171 Spring 2025

Instructor: Maria Samad

Quiz # 3 Solution

QUESTION 1: HASHING (15 marks)

a. A list of numbers is given, and you are required to calculate each number's frequency, i.e., how many times it appears in the given list. Write an algorithm using Hashing that populates the hash table in such a way that the number represents the "key" and its frequency represents the "value". The size of hash table is taken as input from the user. Hash function to be used is Divide Modulo, and collisions must be handled using Quadratic Probing. You may use two separate lists to represent keys and values just like you did in the labs. **YOU ARE NOT ALLOWED TO USE ANY ADDITIONAL DATA STRUCTURES, APART FROM THE ONES SPECIFIED IN THE QUESTION**.

Solution:

- Size = take input from the user
- Keys = [None] * Size
- Values = [None] * Size
- num_lst → given in the question

- for each element, i in num_lst, do the following steps
    - slot_index = i mod Size
    - if Keys[slot_index] == None, do as follows:
        - Keys [slot_index] = i
        - Values [slot_index] 1
    - Else if Keys [slot_index] == i, i.e., the element is already added, then we only need to increase the frequency count in values as follows:
        - Values [slot_index] += 1
    - Otherwise, there might be collision because of slot being already occupied by some other number, so handle collisions as follows:
        - iteration = 1
        - new_index = slot_index
        - while (Keys [new_index] != None and iteration <= Size), resolve collision as follows:
            - new_index = (slot_index + (iteration**2)) mod Size
            - if Keys [new_index] == None, then we have found a new empty slot so update as follows:
                - Keys [new_index] = i
                - Values [new_index] = 1
                - Break
            - Else if Keys [new_index] == i, i.e., the same element matches then increase its frequency in Values list as follows:
                - Values [new_index] += 1
                - Break
            - Otherwise, move to the next slot to find empty spot
                - iteration = iteration + 1

b. Using the algorithm from part (a), perform hashing on the following list of numbers. Assume size of hash table is 9.
- [5, 7, 13, 13, 13, 1, 5, 7, 23, 5, 5, 23, 23, 7, 67, 4, 8, 1, 23, 13, 4, 5, 67, 7, 8, 8, 8]

Solution: Initially hashtable is empty and filled with None

Keys:	None	None	None	None	None	None	None	None	None
Values:	None	None	None	None	None	None	None	None	None

- 5 mod 9 = 5
- 7 mod 9 = 7
- 13 mod 9 = 4
- 1 mod 9 = 1
- 23 mod 9 = 5 → collision, so do quadratic probing:
  - $(5 + 1^2)$ mod 9 = 6 mod 9 = 6, which is free
- 67 mod 9 = 4 → collision, so apply quadratic probing until free slot is found:
  - $(4 + 1^2)$ mod 9 = 5 mod 9 = 5 → collision again, so continue to probe
  - $(4 + 2^2)$ mod 9 = 8 mod 9 = 8, which is free
- 4 mod 9 = 4 → collision, so apply quadratic probing until free slot is found:
  - $(4 + 1^2)$ mod 9 = 5 mod 9 = 5 → collision again, so continue to probe
  - $(4 + 2^2)$ mod 9 = 8 mod 9 = 8 → collision again, so continue to probe
  - $(4 + 3^2)$ mod 9 = 13 mod 9 = 4 → collision again, so continue to probe
  - $(4 + 4^2)$ mod 9 = 20 mod 9 = 2, which is free
- 8 mod 9 = 8 → collision, so apply quadratic probing:
  - $(8 + 1^2)$ mod 9 = 9 mod 9 = 0, which is free

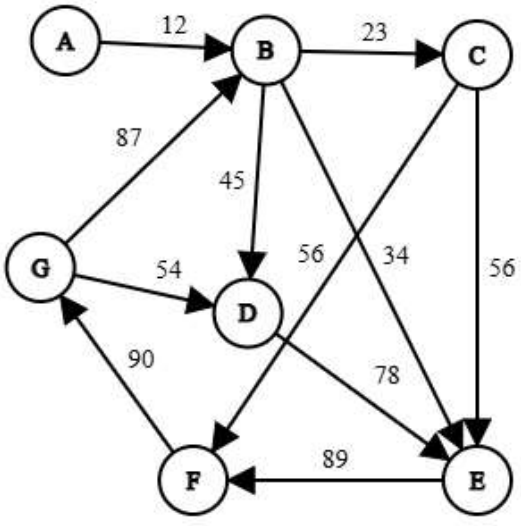
The resultant hashtable:

Keys:	8	1	4	None	13	5	23	7	67
Values:	4	2	2	None	4	5	4	4	2

QUESTION 2: GRAPH TRAVERSAL (10 marks)

For the given graph, apply the Depth-First Search Traversal Algorithm with the starting vertex = A. Show all the working for each step. Construct the corresponding DFS graph, by showing all the tree edges and non-tree edges. Use different notation to represent each edge, and specify a legend for it. Follow ascending order in choosing vertices. Calculate the total cost of traversal in the end.

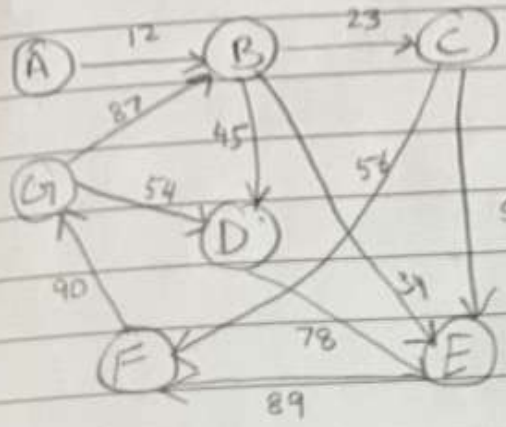
IF YOU ARE MISSING ANY INTERIM CALCULATIONS, YOU WILL NOT GET ANY MARKS



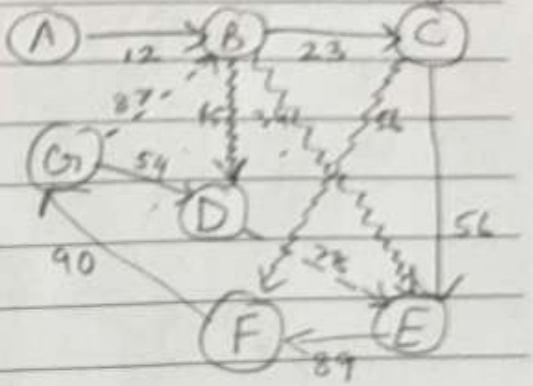
Solution:

Date:

DFS: Start vertex = A



Legend: → Tree edges
- - → Back edges
~~> Forward edges



Stack	Visited list	Path
[B]	[A*]	
[E, D, G]	[A, B*]	A→B? ✓
[E, D, F, E]	[A, B, C*]	B→C? ✓
[E, D, F, F]	[A, B, C, E*]	C→E? ✓
[E, D F, G]	[A, B, C, E, F*]	E→F? ✓
[E, D, F, D, B]	[A, B, C, E, F, G*]	F→G? ✓
[E, D, F, D]	"	G→B? ✗
[E, D, F, E]	[A, B, C, E, F, G, D*]	G→D? ✓
[E, D, F]	[A, B, C, E, F, G, D]	D→E? ✗
[E, D]	"	D→F? ✗
		G→F? ✗
		F→F? ✗
[E]	[A, B, C, E, F, G, D*]	D→D? ✗
[]	[A, B, C, E, F, G, D*]	D→E? ✗

Path: A→B, B→C, C→E, E→F, F→G, G→D

Cost = 12+23+56+89+90+54

= 324

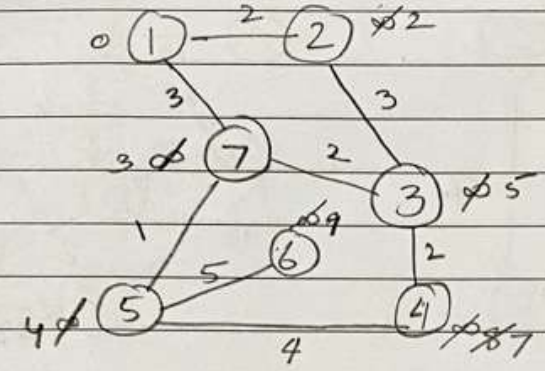
QUESTION 3: SHORTEST PATH (15 marks)

a. Run the Dijkstra's Algorithm on the following undirected, weighted graph, to get the shortest path of every vertex from the starting vertex, 1. Specify shortest path of every vertex from the starting vertex, and calculate the cost of this shortest path. Draw the reduced graph after Dijkstra's algorithm has successfully been implemented, showing the shortest paths between all the vertices. Show your complete working. **IF YOU DO NOT SHOW ANY WORKING, YOU WILL NOT GET MARKS FOR IT**

Solution:

Date:

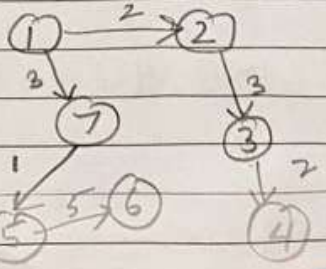
Dijkstra starting vertex = 1



Visited	1	2	3	4	5	6	7
1	0	∞	∞	∞	∞	∞	∞
2		2	∞	∞	∞	∞	3
7			5	∞	∞	∞	3
5			5	∞	4	∞	
3			5	8		9	
4				7		9	
6						9	

Src → Dest	Shortest Path	Cost
1 → 2	2, 1 ⇒ 1 → 2	2
1 → 3	3, 2, 1 ⇒ 1 → 2 → 3	2+3=5
1 → 4	4, 3, 2, 1 ⇒ 1 → 2 → 3 → 4	2+3+2=7
1 → 5	5, 7, 1 ⇒ 1 → 7 → 5	3+1=4
1 → 6	6, 5, 7, 1 ⇒ 1 → 7 → 5 → 6	3+1+5=9
1 → 7	7, 1 ⇒ 1 → 7	3

Reduced graph:



DALMATIAN

b. For the same graph, as given in part (a), define the Graph using Adjacency Matrix Representation

Solution:

```
G(V, E) = [ [ None, 2, None, None, None, None, 3],
            [ 2, None, 3, None, None, None, None],
            [ None, 3, None, 2, None, None, 2],
            [ None, None, 2, None, 4, None, None],
            [ None, None, None, 4, None, 5, 1],
            [ None, None, None, None, 5, None, None],
            [ 3, None, 2, None, 1, None, None] ]
```